```
Output:
Volkswagen Golf GTI
```

Abstract method declarations behave in much the same way as method declarations in interfaces, as seen in the previous section. Just like interfaces, the child class must provide an implementation or trigger a fatal error. Another similarity to interfaces is that only methods can be abstract; there are no abstract properties in PHP.

The one advantage abstract classes have over interfaces is that they can also provide implementations for non-abstract methods. Abstract classes are non-instantiable, but otherwise they are classes in every other respect. Here's a demonstration:

```
abstract class Car
{
    protected static $noise = 'VRRRRM!';

    public function beepHorn(): void
    {
        print "Beep beep!" . PHP_EOL;
    }

    abstract public function makeNoise(): void;
}

class VwGolf extends Car{
    public function makeNoise(): void
    {
        print self::$noise . PHP_EOL;
    }
}

$car = new VwGolf();
$car->makeNoise();
$car->beepHorn();

Output:
VRRRRM!
Beep beep!
```